AME 5101 - Applied Linear Algebra
Practice Exercises

Q1.

The attention algorithm is:

Attention$(\mathbf{X}, \mathbf{Z}, \mathcal{W}_{\text{qkv}}, \text{Mask})$ (tokens are columns)
Input: $\mathbf{X} \in \mathbb{R}^{d_x \times \ell_x}$, $\mathbf{Z} \in \mathbb{R}^{d_z \times \ell_z}$
Parameters: $\mathbf{W}_q \in \mathbb{R}^{d_{\text{attn}} \times d_x}$, $\mathbf{b}_q \in \mathbb{R}^{d_{\text{attn}}}$; $\mathbf{W}_k \in \mathbb{R}^{d_{\text{attn}} \times d_z}$, $\mathbf{b}_k \in \mathbb{R}^{d_{\text{attn}}}$;
$\mathbf{W}_v \in \mathbb{R}^{d_{\text{out}} \times d_z}$, $\mathbf{b}_v \in \mathbb{R}^{d_{\text{out}}}$

1: $\mathbf{Q} \leftarrow \mathbf{W}_q \mathbf{X} + \mathbf{b}_q \mathbf{1}^\top$
2: $\mathbf{K} \leftarrow \mathbf{W}_k \mathbf{Z} + \mathbf{b}_k \mathbf{1}^\top$
3: $\mathbf{V} \leftarrow \mathbf{W}_v \mathbf{Z} + \mathbf{b}_v \mathbf{1}^\top$
4: $\mathbf{S} \leftarrow \mathbf{K}^\top \mathbf{Q}$
5: for each t_z, t_x: if not Mask$[t_z, t_x]$ then $S[t_z, t_x] \leftarrow -\infty$
6: return $\tilde{\mathbf{V}} = \mathbf{V} \cdot \text{softmax}\left(\mathbf{S}/\sqrt{d_{\text{attn}}}\right)$ (softmax applied to each column)

(**a**) Differentiate between the roles of the queries, keys and values.

(**b**) Analyse the purpose of the mask in a decoder-only transformer such as GPT. 5 Marks.

(**c**) Analyse why the scores are divided by $\sqrt{d_{\text{attn}}}$ before the softmax.

(**d**) What would happen to the softmax, and to learning, if they were not?

(**e**) In self-attention with $d_{\text{attn}} = 2$ and $\ell = 3$, the keys and queries are

$$\mathbf{K} = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}, \qquad \mathbf{Q} = \begin{pmatrix} 1 & 2 & 0 \\ 1 & 0 & 1 \end{pmatrix}.$$

Compute $\mathbf{S} = \mathbf{K}^\top \mathbf{Q}$ (line 4), and examine the effect of the causal mask Mask$[t_z, t_x] = [\![t_z \le t_x]\!]$ (line 5) by writing $-\infty$ in the masked entries.

(**f**) Analyse which key receives the larger attention weight for the query at $t_x = 2$, and why.

Q2.

The multi-head attention algorithm is:

MHAttention$(\mathbf{X}, \mathbf{Z}, \mathcal{W}, \text{Mask})$
Parameters: for each head $h \in [H]$, $\mathcal{W}^h_{\text{qkv}}$ as in Attention, with d_{mid} in place of d_{out};
$\mathbf{W}_o \in \mathbb{R}^{d_{\text{out}} \times H d_{\text{mid}}}$, $\mathbf{b}_o \in \mathbb{R}^{d_{\text{out}}}$

1: for $h \in [H]$: $\mathbf{Y}^h \leftarrow \text{Attention}(\mathbf{X}, \mathbf{Z}, \mathcal{W}^h_{\text{qkv}}, \text{Mask})$
2: $\mathbf{Y} \leftarrow [\mathbf{Y}^1; \mathbf{Y}^2; \ldots; \mathbf{Y}^H]$ (stacked vertically)
3: return $\tilde{\mathbf{V}} = \mathbf{W}_o \mathbf{Y} + \mathbf{b}_o \mathbf{1}^\top$

Each of the h heads has width $d_k = d_{\text{model}}/h$.

(**a**) Analyse why a transformer uses several attention heads instead of one. What can several heads capture that a single head cannot?

(**b**) With $H = 2$ heads, $d_{\text{mid}} = 2$, $\ell_x = 2$ and $d_{\text{out}} = 2$, the heads return

$$\mathbf{Y}^1 = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix}, \qquad \mathbf{Y}^2 = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}, \qquad \mathbf{W}_o = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & -1 \end{pmatrix}, \qquad \mathbf{b}_o = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

Trace lines 2 and 3: form $\mathbf{Y}$, determine the shapes of $\mathbf{Y}$ and $\mathbf{W}_o$, and compute $\tilde{\mathbf{V}}$. Which heads contribute to each row of $\tilde{\mathbf{V}}$?

(**c**) Analyse why this keeps the computational cost of multi-head attention close to that of a single head of width d_{model}.

(**d**) Analyse the role of the output projection $\mathbf{W}_o$ in line 3. What would be lost if it were omitted?

(**e**) Analyze what would be lost if MHAttention returned the stacked matrix $\mathbf{Y}$ directly?.

Q3.

A 3-class classifier is trained on three independent examples $(\mathbf{x}_i, c_i)$, $i = 1, 2, 3$. For example i, the network gives the probability vector $\mathbf{q}_i$, and $\mathbf{y}_i$ is the one-hot target of the true class c_i.

The predictions and the targets are

Example	$\mathbf{q}_i$	$\mathbf{y}_i$
1	(0.5, 0.3, 0.2)	(1, 0, 0)
2	(0.1, 0.6, 0.3)	(0, 1, 0)
3	(0.25, 0.25, 0.5)	(0, 0, 1)

Calculate the following quantities:

(a) the probability $P(c_i \mid \mathbf{x}_i, \theta) = \prod_{k=1}^{3} q_{ik}^{y_{ik}}$ of each observed label.

(b) the likelihood of the three examples.

(c) the mean loss $\bar{L}$.

Take $\ln 2 = 0.693$, $\ln 3 = 1.099$ and $\ln 5 = 1.609$.

Q4.

Let $\mathbf{q} = (q_1, \ldots, q_K)$ be the probabilities that a model assigns to K possible outcomes. The surprisal of outcome k under $\mathbf{q}$ is $h_{\mathbf{q}}(k) = -\ln q_k$.

(a) (i) **Show** that the entropy $H(\mathbf{q}) = -\sum_k q_k \ln q_k$ is the average surprisal per draw, when N independent outcomes are drawn from $\mathbf{q}$.

(ii) **Demonstrate** that $H(\mathbf{q}) = \ln K$ when all K outcomes are equally likely, and that $H(\mathbf{q}) = 0$ when one outcome has probability 1.

(b) A model gives the probabilities $\mathbf{q} = (0.5, 0.25, 0.25)$ to three outcomes. **Calculate** the following quantities:

(i) the surprisal of each outcome under $\mathbf{q}$,

(ii) the entropy $H(\mathbf{q})$,

Give the answers in nats. Take $\ln 2 = 0.693$.

Q5.

A classifier is trained on three independent examples. For each example, the classifier gives a probability vector over the classes. The probabilities that it gives to the *correct* class of the three examples are

$$q_{c_1} = 0.5, \qquad q_{c_2} = 0.8, \qquad q_{c_3} = 0.25.$$

(a) Use the idea of surprisal, $h = -\ln P$.

(i) **Show** that the surprisals of two independent outcomes add.

(ii) **Apply** this result to show that the negative log-likelihood (NLL) of N independent examples is the total surprisal of the observed labels.

(b) For the three examples above,

Calculate the following quantities:

(i) the log-likelihood,

(ii) the average cross-entropy loss.

Use natural logarithms and give the answers in nats. Take $\ln 2 = 0.693$ and $\ln 5 = 1.609$.

Q6.

For one example with true class c, the loss in terms of the logits

$$z_1, \dots, z_K$$

is

$$L = -z_c + \ln \sum_j e^{z_j}$$

(**a**) **Show** that this expression follows from $L = -\ln q_c$ and the definition of the softmax.

(**b**) **Show** the identity $\ln \sum_j e^{z_j} = m + \ln \sum_j e^{z_j - m}$, where $m = \max_j z_j$. This essentially demonstrates that the loss can be computed without overflow, even for very large logits.

Q7.

A network with $K = 2$ classes gives the logits $\mathbf{z} = (z_1, z_2)^\mathsf{T}$ for each example.

Three examples have the following logits and true classes:

Example	logits $\mathbf{z}$	true class c
1	$(\ln 3,\ 0)$	1
2	$(0,\ \ln 3)$	1
3	$(\ln 2,\ \ln 2)$	2

(**a**) **Explain** why the logits are not probabilities, and how to convert them to probabilities.

(**b**) **Compute** the probability vector $\mathbf{q}$ for each example, using softmax.

(**c**) **Calculate** the loss $L = -\ln q_c$ of each example, and the mean loss $\bar{L}$ over the three examples.

Take $\ln 2 = 0.693$ and $\ln 3 = 1.099$.

Q8.

A classification data set has n independent examples:

$$\mathcal{D} = \{(\mathbf{x}_1, c_1),\ (\mathbf{x}_2, c_2),\ \dots,\ (\mathbf{x}_n, c_n)\}.$$

Here $c_i \in \{1, \dots, K\}$ is the true class of the input $\mathbf{x}_i$, and K is the number of classes. For example i, the network gives the probability vector $\mathbf{q}_i = \text{softmax}(\mathbf{z}_i)$ with entries q_{ik}. The one-hot target $\mathbf{y}_i$ has $y_{ik} = 1$ if $k = c_i$ and $y_{ik} = 0$ otherwise. Let θ denote the parameters of the network.

(**a**) **Show**, using the one-hot target, that

$$P(c_i \mid \mathbf{x}_i, \theta) = \prod_{k=1}^{K} q_{ik}^{y_{ik}}.$$

(**b**) **Demonstrate** that this gives the mean loss

$$\bar{L} = \frac{1}{n} \sum_{i=1}^{n} (-\ln q_{ic_i}),$$

where q_{ic_i} is the entry of $\mathbf{q}_i$ for the true class c_i.

Q9.

A network with $K = 3$ classes has the logits $\mathbf{z} = (\ln 2,\ \ln 2,\ 0)^\mathsf{T}$, and the target is $\mathbf{p} = (0.5,\ 0.5,\ 0)^\mathsf{T}$.

(**a**) **Calculate** the probability vector $\mathbf{q}$ and the loss $L = H(\mathbf{p}, \mathbf{q})$

(**b**) **Compute** the gradient $\mathbf{q} - \mathbf{p}$.

Take $\ln 2 = 0.693$ and $\ln 5 = 1.609$.

Q10.

A network has K output classes. Its small random weights make all the logits $z_1, \dots, z_K$ close to zero at the start of training.

(**a**) **Show** that if all the logits are equal, the softmax gives $q_k = 1/K$ for every class k.

(**b**) **Apply** this result to show that the loss $L = -\ln q_c$ is equal to $\ln K$, whichever class c is the true class.

Q11.

For one example with true class c and logits $z_1, \dots, z_K$, the loss is

$$L = -z_c + \ln \sum_{j=1}^{K} e^{z_j}.$$

Let $\mathbf{q} = \text{softmax}(\mathbf{z})$, and let $\mathbf{y}$ be the one-hot target.

(**a**) **Apply** the chain rule to differentiate L with respect to a logit z_k. 5 Marks.

(**b**) **Show** that $\partial L/\partial z_k = q_k - y_k$. 5 Marks.

Q12.

Consider layer l of a network. Its pre-activations are

$$z_i^{(l)} = \sum_j W_{ij}^{(l)}\, a_j^{(l-1)} + b_i^{(l)}.$$

Suppose that $\boldsymbol{\delta}^{(l)} = \partial L/\partial \mathbf{z}^{(l)}$ is known.

(**a**) **Apply** the chain rule to obtain $\partial L/\partial W_{ij}^{(l)}$ and $\partial L/\partial b_i^{(l)}$ in terms of $\delta_i^{(l)}$ and $a_j^{(l-1)}$. 5 Marks.

(**b**) **Use** these results to write $\nabla_{\mathbf{W}^{(l)}} L$ as an outer product.